

Transport System.

1) Void SmartCity::buildTransport(Graph G)

↳ Basically it is our main function handling all the graph creation & subgraphs extraction logic.

• Total time complexities

→ Initialize nodes $O(\text{total Nodes})$

→ Register nodes $O(\text{total Nodes})$

→ Connect bus routes $O(\text{buses} \times \text{avg Route})$

→ Connects stops $O(\text{stops}^3)$

↳ as here we connect our 3

nearest stops & for which we also use bubble sort to sort the stops which leads it $O(\text{stops}^3)$.

→ For hospitals, schools, malls & facilities also $O(n \times \text{stops}^2)$

→ Now for extracting all b sub graphs Medical, Education, Commercial, Facilities, Transport we get $O(\text{total Nodes} + \text{total Edges})$

↳ $O(\text{stops}^3 + \text{total Nodes} + \text{total Edges})$ as stops are largest component & $\text{stops}^3 > \text{total Nodes}$
Worst case time complexity = $O(\text{stops}^3)$

• Total space complexity.

For all subgraphs = $O(\text{Nodes} + \text{Edges})$

For registering types = $O(\text{total Nodes})$

All Edges = $O(\text{total Nodes})$

So worst case space complexity will be $O(\text{total Nodes})$.

2) dijkstra()

→ For initialising visited, parent & distance array and marking visited and validating node

↳ $O(\text{total Nodes})$.

→ For linear search to find minimum = $O(\text{total Nodes}^2)$

↳ worst time complexity = $O(\text{total Nodes}^2)$.

- Space complexity.

visited, distance & parent arrays will be having space of total Nodes only so

↳ worst space complexity = $O(\text{total Nodes})$.

3) Shortest Path Between Stops()

- Time Complexities

→ Initialising arrays = $O(N)$

→ Display distance, time, summary = $O(1)$

→ Running Dijkstra = $O(\text{total Nodes}^2)$

↳ worst time complexity = $O(\text{total Nodes}^2)$

- Worst space complexity is $O(\text{total Nodes})$ as it is for initialising distance, parent, visited arrays & also traversing them afterwards so

↳ worst space complexity = $O(\text{total Nodes})$

4) Find Nearest Hospital()

- ^{Time}Space complexities:

→ Initialising arrays = $O(\text{total Nodes})$

→ Dijkstra & Sorting hospitals = $O(\text{total nodes}^2) + O(\text{hospitals}^2)$

→ Extract distance & heap = $O(\text{hospitals} \times \log \text{hospitals})$

↳ So worst case time complexity will be $O(\text{total nodes}^2)$

- Worst space complexity is same as $O(\text{total nodes})$ as using distance, parent & visited array which traverses whole graph.

5. FindNearestBusToPassenger()

→ for get passenger location, initializing, display all have $O(1)$ complexity

→ Loop through buses = $O(\text{buses})$

so worst time complexity = $O(\text{buses})$

- Space complexity.

$O(1)$ is worst space complexity as no new variables or array is made just using local ones only.

6. Displaybus Route()

↳ worst time complexity = $O(\text{route length})$

↳ as simply we traverse through linked list of route only in this function.

- Space complexity.

Worst space complexity = $O(1)$ as we ^{already} just visit the created linked list ~~no new~~ first variable created so $O(1)$

7. addStopToBusRoute()

• Time complexities.

→ checking bus, stop exists or not, getting input from users = $O(1)$

→ Displaying buses = $O(\text{buses})$

→ Displaying current route = $O(\text{route length})$

→ check for duplicate = $O(\text{route length})$

→ Display all stops = $O(\text{route length})$

↳ worst time complexity = $O(\text{buses} + \text{stops} + \text{route length})$

• Space complexity.

↳ $O(1)$ as again no new array or anything was made just variables were made.

6. addPassengerToQueue()

Worst time complexity = $O(1)$

↳ as we just gets input, add at rear of queue so simple $O(1)$

• Space complexity

$O(1)$

↳ as we just creates variables only, no new allocations.

9. displayPassengerQueue():

→ worst time complexity = $O(\text{queue size})$ as it traverses through whole queue.

• Space complexity.

↳ $O(1)$ as only count variables were created in here nothing else.

DATE: ___/___/___

10. Simulate Passenger Boarding()

→ Displaying & dequeue operations = $O(1)$

→ loop numofBoard times to Board that
number of passengers = $O(\text{numofBoard})$

→ Displaying queue = $O(\text{queue size})$

↳ Worst Time complexity = $O(\text{queue size})$

↳ as queue size will be $\geq \text{numofBoard}$

- Space complexity Same $O(1)$ as nothing new declared except simple variables.

11. trackRouteHistory()

→ Worst time complexity = $O(1)$

↳ as we just push previous stop to stack.

→ Worst space complexity = $O(1)$

↳ Nothing new formed.

12. displayRouteHistory():

→ Worst time complexity = $O(\text{stack size})$

↳ as loops through whole stack

→ Worst space complexity = $O(1)$

↳ No new array or something was created.

Medical System.

searchHospital by Specialization()

1. buildMedicalSubgraph()

loop hospitals = $O(\text{hospitals})$

substring search = $O(\text{speclength} \times \text{searchlength})$

↳ but mainly it will be $O(\text{hospitals})$ as it depends on them how many of hospitals specialization array we will be needing to search.

→ worst space complexity.

↳ $O(1)$

as declaring of array, variables and all are done only single time means space is not kept being expanding.

↳ constant space.

2. buildHospitalPriorityQueue.

loop number of hospital times = $O(\text{hospitals})$

Insert into heap = $O(\log \text{ hospitals})$ per insert

so

↳ worst time complexity = $O(H \log H)$

• worst space complexity.

↳ $O(\text{hospitalcount})$ as heap array of that size will be created

3. displayTopHospitals()

so for build heap = $O(H \log H)$

↳ find before

further finding hospitals with bed heap
returns = $O(\text{Hospitals time})$

Loop Hospital time = $O(H)$ to search
each hospital in venue case.

so, worst time complexity = $O(H^2)$

as first to search hospital then specification
in that hospitals. & if $n = \text{hospitals that}$
leads to $O(H^2)$

• Space complexity.

↳ $O(\text{hospitals})$

↳ as we declare buildHeap[] array

& struct bedData[] array so worst

space complexity = $O(\text{hospitals})$ means linear.

4. Search Medicine By Name()

↳ using hash functions so

↳ worst time complexities = $O(1)$

• ↳ space complexity = $O(1)$

5. Search Medicine By Formula()

↳ Use hash function to find formula

worst time complexities = $O(1)$

↳ worst space complexity = $O(1)$

6. Compare Medicine Prices()

- Time complexity = $O(\text{pharmacies} + \text{count}^2)$
 $\rightarrow$ as we sort number of pharmacies by price means those which have these medicines.
 $\rightarrow O(\text{pharmacies})$ as we search all pharmacies to find matching medicine.

- space complexity.
 $O(\text{pharmacies})$
 $\rightarrow$ as we form medName & PriceList array equal to pharmacies number considering each will have $O(\text{pharmacies})$

7. dischargePatient()

$O(1)$ worst time complexity.

$\rightarrow$ as we look for patient using Hash table.

$O(1)$

$\rightarrow$ same constant space and minimal as using hash table.

8. addDoctor()

$\rightarrow O(1)$ worst time complexity.

$\rightarrow$ as we use hash functions to store doctor, check duplicate, and ^{find hospital} so $O(1)$ time complexity.

- Space complexity = $O(1)$ same as constant memory is being used.

DATE: ___/___/___

9. addPatient()

<sup>Worst
Time
Complexity</sup> $O(1)$ as using hash functions for
everything validating, inserting and all.
Space complexity $O(1)$

↳ Constant space being used.

10. assignPatientToDoctor()

→ Worst case time complexity.

↳ $O(1)$ as finding patient & doctor
both by hash functions & using them

→ Worst case space complexity.

↳ $O(1)$ as constant space being used.

Public Facilities (mosque, parks, ...)

id, facility [10], name_{maxlength}, type, location

1. load Public Facilities

check exception if you load some yourself (later)

$O(n)$ - each facilities entry once.

2. display facilities

$O(n)$ iterating through facilities 3x

3. adding the facilities

$O(1)$ direct insertion

4. finding facility by type.

$O(V^2)$ for dist. $V = \text{total nodes}$. $O(n)$ to filter by type

5. finding facility by name.

$O(n)$ worst case if all keys collide

6. Public facility menu

$O(1)$.

helpers

manual Atof:

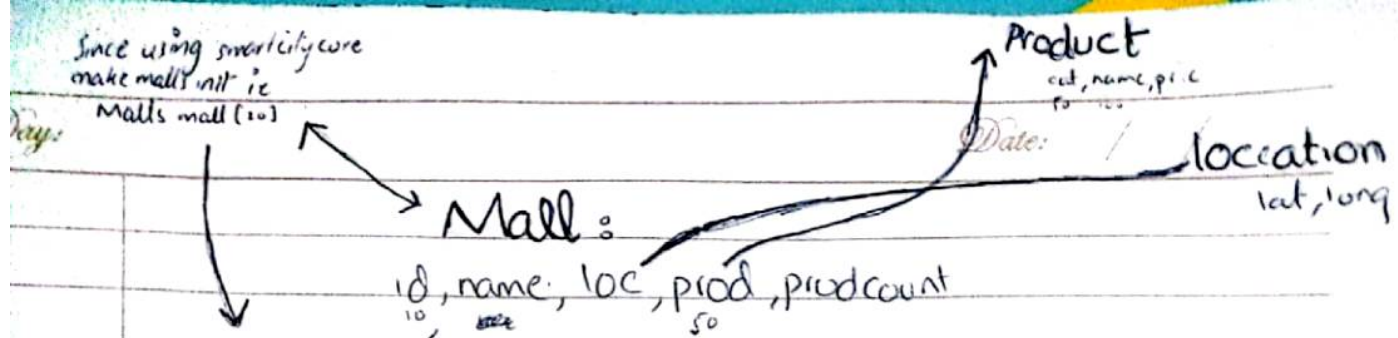
$O(n)$ $n = \text{length of string}$

converting string to floating point

hash: $\Theta(n)$

string to int keys.

using polynomial rolling hashing



1. loading the mall data from the file $O(n)$

exceptions file not opening.

some test malls if file not available

reading each line as per given in the instructions ID, name, lat, long

each mall is then stored in array, inc count

initially no products

Basic functions as per instructions given in questions

2. display the mall $O(n)$

3. add a new mall $n = 1, 100$

check array boundaries

check if it already exists

id $\rightarrow$ hash table

$O(n)$ checks for dup
ID by scanning malls

4. add products to a mall

check for the boundaries

product key $\rightarrow$ hash

if via hash $O(1)$
otherwise $O(n)$

5. search product by name

user input

use product count for loops and search products

$O(n \times m)$
 $n = \text{mall count}$
 $m = \text{products per mall}$

6. finding nearest mall -

search via dist slopes.

$O(V^2)$ via numerical mapping nodes into
dijkstra
 $+ O(V)$ for min dist

7. Product by category based

search malls $\rightarrow$ products $\rightarrow$ categories.

check boundaries

8. lastly MENU. $O(1)$

School Class

(1) load(Education data)

↳ linear CSU reading

$O(N)$ = where n is number of schools

↳ All school record is processed once and operations like string parsing run in constant time.

// N records so atleast N time required.

2) build school system tree() // display also has the same nary tree structure. // 20 children. Tree used

each school child of root 3 level n ary

each school has 3 depth tree.

each dept has 4 classes

$O(N)$

3) build school Rank() // min heap is used.

// Since we add then heapify,

Binary tree so height traverse = $\log S$

we check for all elements = S

$S \times \log S$

$\Rightarrow S \log(S)$

Find nearest school

basic dijkstra use without priority queue

Initialization $\rightarrow$ Runs N times

Run dijkstra of city graph. $\Rightarrow O(N^2)$

Build path by walk $\Rightarrow O(N)$ worst-case

$$O(N) + O(N^2) + O(N)$$

$$\Rightarrow O(N^2)$$

Population

1) build Population tree

N of tree

Time complexity in $N \Rightarrow$ Number of citizens.

4 levels

(1) Sector

(2) Street

(3) House Number

(4) Person.

} all are nodes

We are having linear tree but independent
so it is $O(N)$

2) find nearest bus to passenger.

b is the number of buses we added in the original heap.

// compute distance is like a mathematical function $O(1)$,

$O(b)$

⇒ find citizen by CNIC

$O(k)$ where k is length of the string

// in searching $O(n)$ worst case → compute bucket index.

↳ all keys in the same bucket

so $O(k) + O(n) \Rightarrow O(k)$.